SMU
SINGAPORE MANAGEMENT UNIVERSITY

School of
Computing and
Information Systems

IS212 Project Slide Template

SMU SINGAPORE MANAGEMENT UNIVERSITY
School of Computing and Information Systems

Index

This page acts as a “content page” for your project.

Item	URL, location or file/folder name in zip file
Product backlog	1/product-backlog.xlsx
Git URL (e.g. GitHub) *(remember to add your two instructors to the repository; see instructions)*	https://github.com/MochaLxtte/ConnectSphere
Exports of project management tools (e.g. Google Sheets, Jira, Notion docs, …)	1/product-backlog.xlsx
Video recording URLs of sprint 4 scrum ceremonies	https://www.youtube.com/playlist?list=PLTmUzhbdJbVo

Ground Rules

- **Do not add or delete any slide or question.** If something does not apply, state “Not applicable” and explain why.
- **Do not answer only with general statements.** Cite evidence where appropriate or requested, tell us exactly where the relevant Jira item, pull request, commit, test, document, meeting record, or other artefact is.
- **Disclose AI use accurately.** AI use itself is not penalised.
- **Any team member may be asked about any answer in the deck during the Q&A.**
- **Claims made in the deck WILL be verified against the submitted repository, backlog and Scrum artefacts.**
- **Known shortcomings should be disclosed rather than hidden.**
- You are allowed to change the **sizes** of the given text boxes, but not delete them. (You decide how much you want to say for each item, without exceeding the slide or making the text unreadable.)

Handling the Customer Change (i)

- Which of the changes in Week 7 had the **most** impact and why?

Change 1, having to additionally factoring in venue setup and turnaround time into the venue booking was the most broadly impactful and required us to reopen quite a few stories for subsequent sprints.

- Describe how you handled these changes, and which components of your project were affected and how.

These are the components that were affected:

1. Venue catalogue and data - staff need to set and save each venue's buffer values. We implemented that existing venues receive defaults, and adding the 2 buffer periods caused bookings already on the calendar to clash.
2. Search and availability - available times given must be additionally checked against buffered bookings, operating hours and operational blocks.
3. Booking and approval - the buffer periods are also factored into the conflict checks
4. Calendar, suitability and staff views - these need to reflect the effective occupied period and flag existing bookings that now overlap.
5. Tests and design records - more test cases for whether the additional buffer times caused overlaps, and concurrent approvals, alongside regression checks for existing access and booking behavior.

Handling the Customer Change (ii)

- Which of the changes in Week 7 had the **least** impact and why?

Change 3, multiple venues for one event, was least impactful relative to the others. The system could be easily extended to support multiple venue bookings for one event

- Describe how you handled these changes, and which components of your project were affected and how.

The main fix we had to make was ensuring that cancelling or replacing one arrangement did not affect the others. These were the areas we changed to handle this change:

1. Booking records and relationships - each venue arrangement remains associated with the same event and can be identified separately.
2. Coordinator actions and API - cancel one arrangement or request a replacement for it without applying an event-wide change.
3. Event and booking views - show the separate arrangements and their replacement relationships.
4. Suitability and conflict checks - assess a saved arrangement using its own venue and time, while checking conflicts appropriately.
5. Tests - verify that cancelling or replacing one venue leaves the other arrangement unchanged, including authorization and failure cases.

SMU Singapore Management University

School of Computing and Information Systems

CFO Update

Provide THREE concrete examples from your project where you added value on top of what AI did:

Example	What AI Could / Did Do	What Your Team Added	Why Did This Add Value?	Evidence
1	Wrote the “cancel or replace one venue” feature and its tests, where all automated checks passed.	A teammate’s code review spotted that two actions could freeze each other if run at the same time. We proved it with a failing test, then fixed it.	We prevented a bug that would have crashed the app when two users acted at once.	PR #67 review and fix commits
2	Built the safety review feature. All automated checks passed.	TA review against the requirements found two gaps. The owner chose the safer of two possible fixes and recorded why.	Without it and not intervening on what AI did, a safety approval could stay valid after the venue changed, which may jeopardise event participants if glazed over.	PR #78 review; decision log (D9–D10)
3	Built the venue hold and outage features, then flagged that two requirements contradicted each other and suggested options.	The Product Owner decided how the product should behave, explained why, and updated the requirements and tests to match.	We provided human judgement on design decisions, that optimally fit the customer needs that AI might not conceive of.	PR #58; PO decision record (25 Oct)

SMU SINGAPORE MANAGEMENT UNIVERSITY
School of Computing and Information Systems

AI Usage

- Did using AI cause any rework? If so, what happened?

Yes. AI went beyond was we instructed and built unplanned features while setting up the database, and we had to remove them. We then added the AGENTS.md guardrail: AI must propose a plan and wait for our approval.

- Which part of the final system contains the clearest evidence that your team contributed more than simply prompting an AI tool? Why?

The venue booking and safety rules. The AI's code passed every test, but our reviews still found real bugs.
- PR #77 Tristan's review : "CI passes, but venue turnaround buffers can create booking conflicts"

- What decision required human judgement rather than simply code generation? Why?

Moving a requirement to the stories that would build what it needs, instead of dropping it (KAN-81). Deciding where work belongs across people and sprints is a planning call, not a coding task.

"The most important value our team added beyond AI was was checking AI's work was actually right, because AI code could pass every test and still be wrong."

SMU SINGAPORE MANAGEMENT UNIVERSITY

School of Computing and Information Systems

Controlling what AI produced

For each of the points below, explain what you did when using LLMs:

- Delegate

We used LLMs to help with defined tasks such as drafting documentation, creating code, and suggesting test scenarios. The team remained responsible for deciding scope, estimates, and acceptance;

- Constrain

We gave the LLM project-specific instructions in AGENTS.md, including the required story workflow, document order, testing expectations, Git practices, and rules not to invent decisions or test results. We also supplied the relevant story criteria and project records to keep assistance within the agreed scope.

- Verify

We checked LLM-assisted work against Jira criteria, repository records, code, and test results. In addition, we also did manual testing to judge the workflow created by the LLM. Changes were reviewed through the project's branch and pull-request process, and CI ran automated checks. We recorded limitations where evidence was incomplete so a passing automated suite was not treated as proof of integrated acceptance, and checked against the Definition of Done.

- Own

Team members remained accountable for the code, documentation, and decisions produced with LLM assistance. The PR template requires authors to disclose which parts were AI-assisted and how they checked them, and members are accountable for explaining their work and decisions.

Git Strategy

How did your team use Git to support collaboration throughout the project, rather than simply using it as a place to store code? We are particularly interested in whether your Git strategy evolved over the project, how consistently it was followed, and how it worked together with your CI process to ensure that changes were reviewed and verified before being integrated. Note: Your branching, pull request, review and merge practices should help the team work in parallel, reduce integration problems, and protect the stability of the main branch.

We used Git throughout the project to coordinate work, not only to store code. We worked on story-specific branches so team members could develop in parallel, then used pull requests to bring changes together and allow review. Our process became more formal over time, with guidance to branch from the latest main, link commits to story IDs, describe the acceptance criteria addressed, and report the checks run.

Our CI process complemented pull requests by automatically running project checks, coverage, database tests and browser tests on pull requests and pushes to main. This gave reviewers test results to consider alongside the code and helped identify obvious issues before integration. We have evidence of review feedback being addressed and CI passing for pull requests.

SMU SINGAPORE MANAGEMENT UNIVERSITY

School of Computing and Information Systems

Testing Strategy (i)

Metric	Result
Number of manual e2e test cases using Wk4 template	24 manual e2e test cases
Number of unit tests	3,397 test cases: 1,363 frontend + 2,032 backend + 2 browser-environment guards
Number of integration tests (if any)	836 integration test cases
Number of automated e2e tests (if any)	253 Chromium tests, run twice with zero retries. Mostly API/auth-substituted, so not full live journeys
Coverage (explain why if not 100%)	**Not 100%.** Frontend: 96.56% stmts, 93.41% branches, 98.54% funcs, 98.92% lines. Backend with PostgreSQL: 99.24 / 97.88 / 99.72 / 99.46%. Some of our final coverage gaps involve startup and root-component code already exercised by browser tests, whose execution is not merged into the unit coverage report. Others involve an unused placeholder component and an unused backend helper that do not participate in current workflows. Optional display variations, such as missing venue descriptions or partial opening-hours labels, also remain uncovered. Additional tests for these paths would provide limited confidence compared with tests for booking conflicts, permissions and data integrity. These are reasonable exceptions; the remaining access-denial and rollback-error paths should still be acknowledged as testing gaps.
Current passing tests	4,486 on final run
Current failing tests	0 currently
CI status	Pass on latest and final merge.
Number of known unresolved defects	0

- Which type of testing provided the **least** value to your team, and why?

Browser tests using simulated API responses provided the least value. They checked how the interface behaved, but did not confirm that the app worked with the real API and database. Although CI passed, later testing with real login and the shared database found three issues. This shows why we also needed connected testing and database tests. MAN-20261101-02 (real login and shared database; browser operated by Codex) found 3 regressions (TC-06-446, TC-06-447, TC-15-108). Connected runs and real PostgreSQL tests found more of what mattered.

Testing Strategy (ii)

Describe:

- How you identified edge-cases in testing

We identified edge cases by translating each story's acceptance criteria into test scenarios and checking normal, boundary, conflict, and failure conditions. For example, the venue-buffer change led us to test zero and maximum buffer values, exact time adjacency, overlapping bookings, changes that affect existing bookings, and concurrent approvals. We also used the test specifications and traceability records to link acceptance criteria to cases and test files.

- How the customer change and/or CFO statement affected testing

The Week 7 customer changes expanded and redirected testing. We added cases for the new behaviors while retaining checks for the original requirements. For example, change 1 required tests across venue search, availability, suitability, booking and approval. We planned checks for all six changes. We also did more manual testing across the whole team to ensure that user stories were working as we expected.

- Has every test case been verified by a human? If not, why? If so, what was the process/challenges?

We did not manually verify every test case or review every behavior against every acceptance criterion. Given the size of the automated suite, we relied on automated checks and CI for broad coverage, while manually checking selected end-to-end workflows and focused retests using the running application. We also addressed issues raised during some pull-request reviews. Therefore, our verification was meaningful and selective.

CI Pipeline (i)

- **Complete:**

CI Question	Your Team's Answer
What events trigger your CI pipeline?	On pull request and merge to main
What checks/tests are run automatically?	Unit tests, integration tests, automated end to end tests
Is CI required to pass before code can be merged into main?	Yes, excluding special cases
How many CI failures (CI going red) did your team encounter during the project?	19 (https://github.com/MochaLxtte/ConnectSphere/actions?query=is%3Afailure)
What was the most meaningful CI failure you encountered?	https://github.com/MochaLxtte/ConnectSphere/pull/63 Integration failure due to conflict with another PR
What caused it, and how was it resolved?	A conflict in database tables caused by working simultaneously on features requiring similar database tables
Did CI ever prevent faulty code from being merged? Give one example.	Yes (https://github.com/MochaLxtte/ConnectSphere/pull/63)
Did you ever need to bypass CI (e.g. git force push / move code from local main to local branch)? Why? What did you do?	Yes (https://github.com/MochaLxtte/ConnectSphere/pull/63). Another pull request was merged first to change the existing database tables, then a git rebase was needed to use the same tables

CI Pipeline (ii)

- What are the most interesting things in your CI Pipeline that you wish to highlight? Why? Where did you get the idea from, and what benefits do you get from this?

1. We used a real database in every CI run. A PostgreSQL 17 service runs in CI, and migrations and seed apply from empty before the tests. Benefit: it catches what mocks cannot, such as the 11 failures in PR #51 and migration-history failures in PR #63.

2. We kept evidence for all failed runs. Coverage and Playwright reports upload with if: always(), and we retain key logs under docs/scrum/*/evidence, which reaped the benefit of testing records pointing at exact runs and commits (main-ci.json). GitHub only keeps these artifacts for 14 days, hence the repository copies.

3. We had newer pushes cancel older runs. Concurrency cancel-in-progress cancelled any superseded runs which reaped the benefit of faster feedback and better integration efficiency.

Where the ideas came from: the course's Week 9 testing and CI slides are our recorded reference (docs/testing/README.md). The stricter items grew from our own failures, with AI assistance; for example, combined coverage gates arrived in PR #80 during release hardening.

SMU SINGAPORE MANAGEMENT UNIVERSITY
School of Computing and Information Systems

CI Pipeline (iii)

Show ONE concrete example of CI providing value:

- **Change being integrated:**
  What feature/fix was the team trying to merge?

KAN-34 : Connect draft - editing UI to the shred event API

- **What failed:**
  Which automated check or test failed?

Npm run test : e2e (event-requests.spec.ts)

- **Why it failed:**
  What problem in the implementation was detected?

Test expected `/events/${request.id}` pushed `/events/${request.id}/edit`

- **What the team did:**
  How did you resolve the issue?

Edited the code such that if in drafting phase, use API `/events/${request.id}/edit` ,while every other status leads to regular API `/events/${request.id}`

SMU SINGAPORE MANAGEMENT UNIVERSITY

School of Computing and Information Systems

Team Contribution and Ownership

Team Member	Main Areas Worked On	Approx. Contribution	Code/PR Examples	Other Contributions
Arvin	Database, buffers, registration, notifications	16.7%	PR #48, #70	Scrum Master; WS2 docs
Kai Jie	Drafts, bookings, outages, safety review	16.7%	PR #47, #78	WS3 testing docs
Tristan	Login, request review, coordinator assignment, equipment	16.7%	PR #34, #62	Integration lead; code reviews; CI
Irfan	Venue catalogue, holds, change requests	16.7%	PR #53, #65	Product Owner; AGENTS.md; WS2 review
Ernest	Stock maintenance, registration, notifications,	16.7%	PR #74, #76	Slides
Ryan	Login, venue search, cancellation	16.7%	PR #26, #56	WS1 backlog docs

Anything Else?

- Anything else you would like us to know about your project, process, or otherwise? (It's OK to put "N/A" here.)

N/A

End of Submission

No further slides beyond this point.